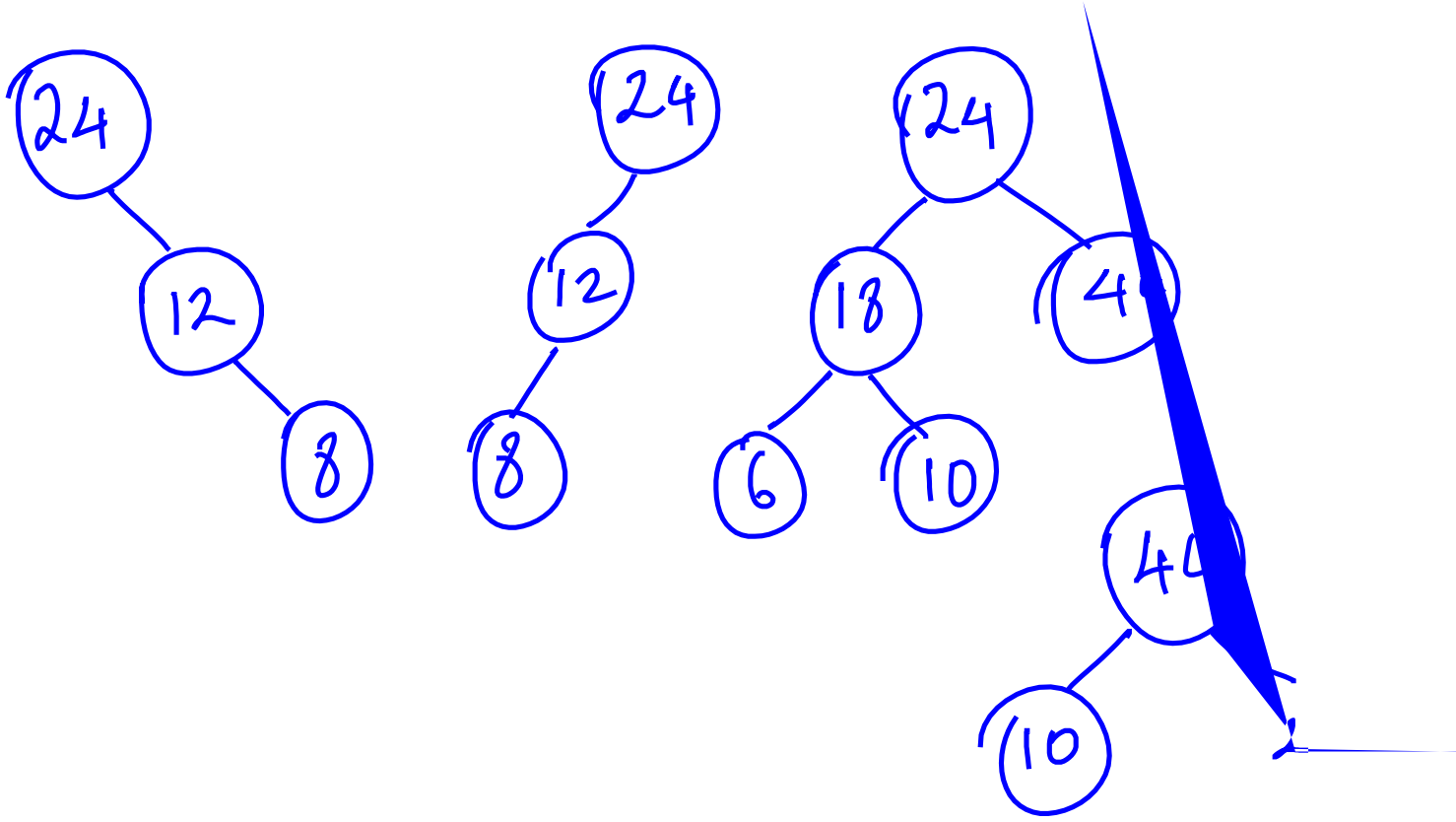
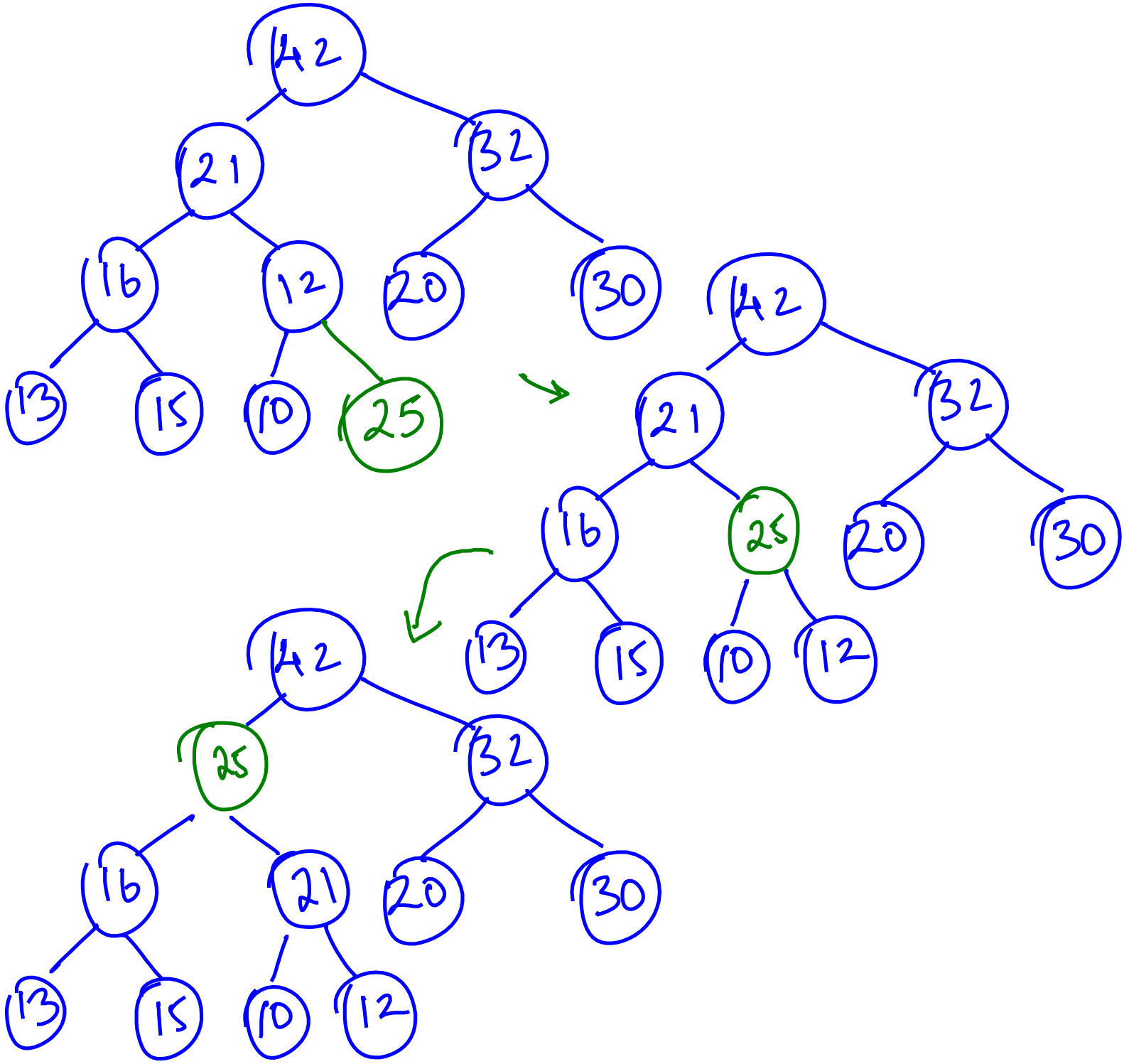
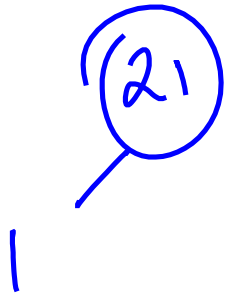

24
12
8
24
12
8
24
18
6
10
10

Reheap UP


42
21
32
16
12
20
30
13
15
10
25
42
21
32
16
25
20
30
13
15
10
12
42
25
32
16
21
20
30
13
15
10
12

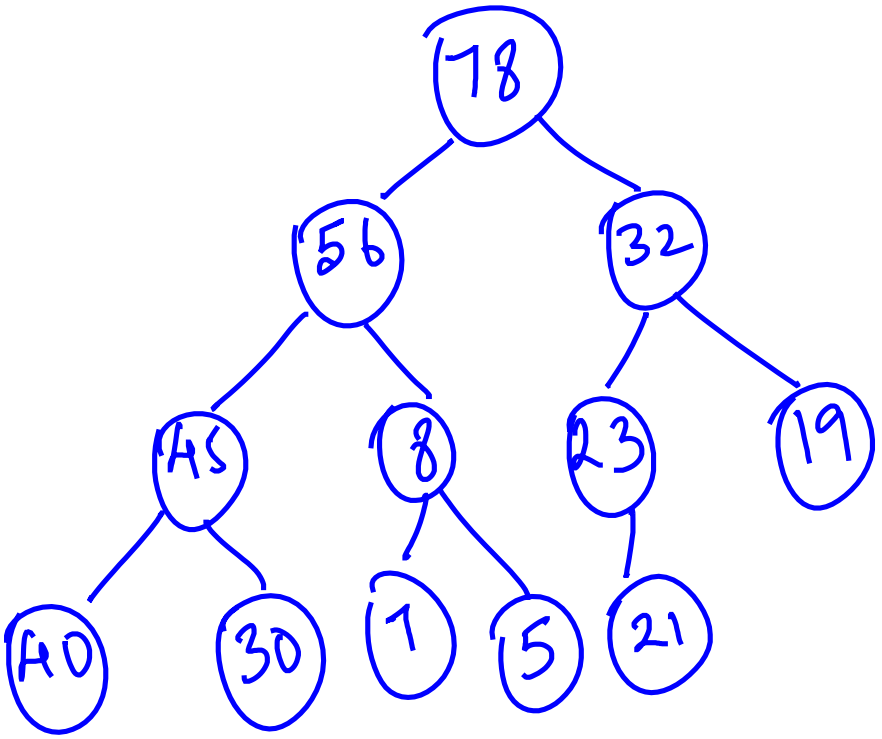
Recap Down



Sai.

Implementation idea

→ Let us build a heap ADT using Arrays.



arr:

0	1	2	3	4	5	6	7	8	9	10	11
78	56	32	45	8	23	19	40	30	1	5	21

Parent Left child

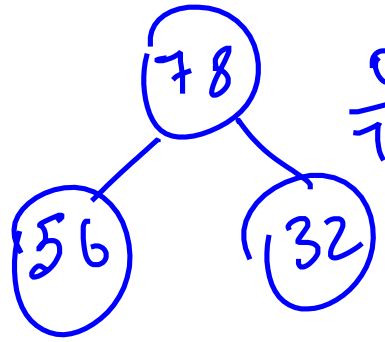
III.

Node	Parent
1	0
2	0
3	1
4	1
5	2
6	2
i	?

IV.

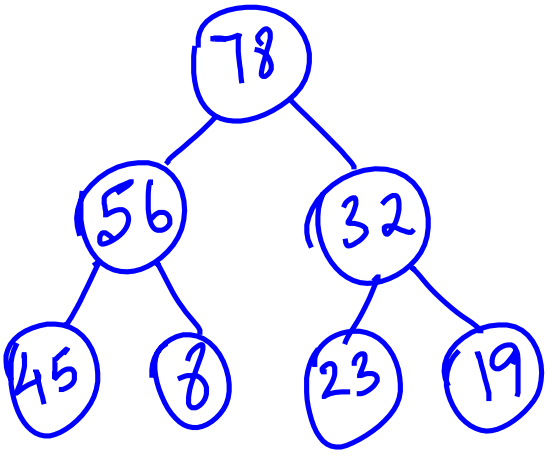
0
78

Nodes = 1, Leaf starts at 0.



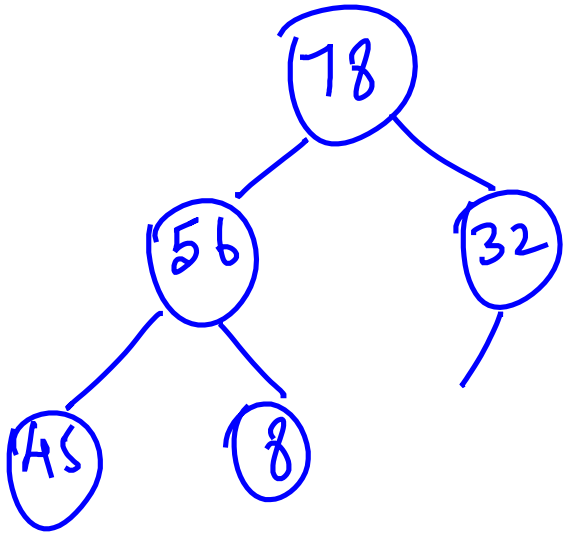
0	1	2
78	56	32

Nodes = 3, Leaf starts at 1.



0	1	2	3	4	5	6
78	56	32	45	8	23	19

Nodes = 7, Leaf starts at 3.



Sai

Implementation:

→ Heapify : Given an array, Convert it into a heap ADT.

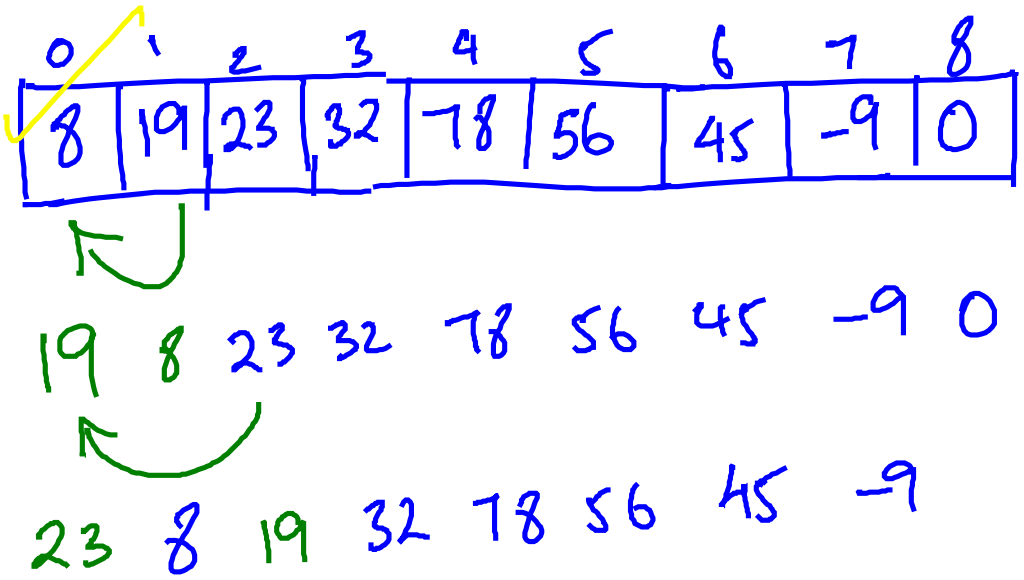
→ Insert : Insert an element into the heap.

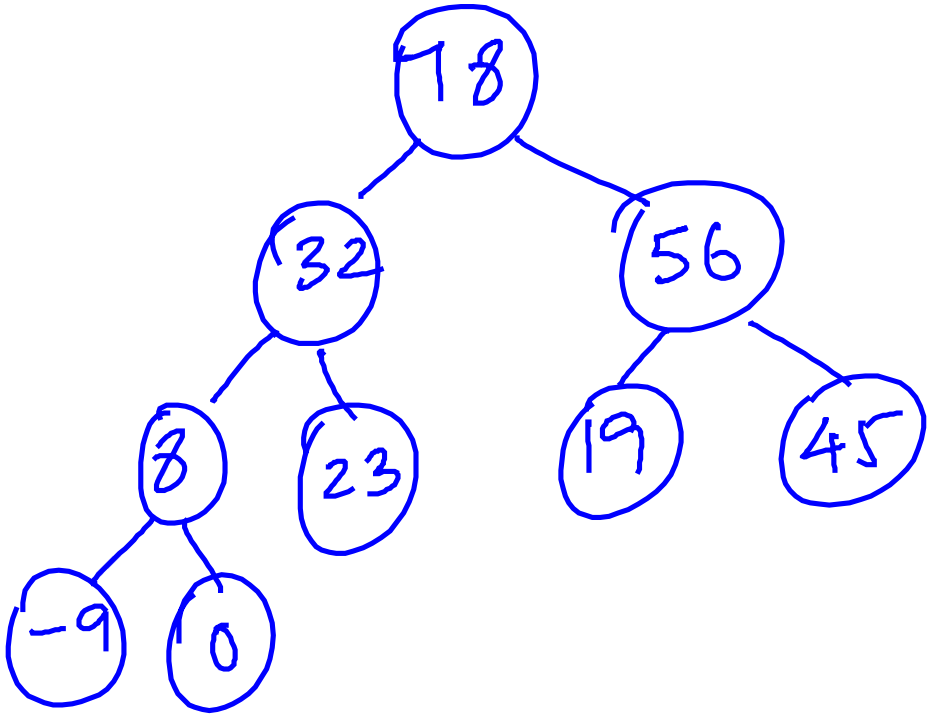
→ Delete : Delete an element from the heap

How will main() look like?

```
int main()
{
    int arr[100] = {8, 19, 23, 32, 78,
                    56, 45, -
```

Heapify :


0 1 2 3 4 5 6 7 8
8 19 23 32 78 56 45 -9 0
19 8 23 32 78 56 45 -9 0
23 8 19 32 78 56 45 -9


18
32
56
8
23
19
45
-9
0

```
void heapify (int arr[], int N)
{
    // first element forms a heap, by default
    // insert 8
    int i = 1

    for (i = 1 ; i < N ; i = i + 1)
    {
```

```
void insert ( int ar[], int *p, int c,
                                int n)
{   // add n into the heap in ar
    if ( *p == c)
    {
        printf ("Heap is full. Element not
                                added\n");
        return;
    }
    ar[*p] = n;

    j = *p;
    while ((j-1)/2 >= 0 &&
                ar[(j-1)/2] < ar[j])
    {   swap(&ar[j], &ar[(j-1)/2]);
        j = (j-1)/2;
    }

    *p = *p + 1;
}
```

```
int delete(int arr[], int *p)
{
    int r = arr[0];
    int j = 0, k = 0;
    arr[0] = arr[*p - 1];
    while(2*j + 1 < *p)
    {
        k = 2*j + 1;
        if(2*j + 2 < *p &&
           arr[2*j+2] > arr[2*j+1])
            k = 2*j + 2;
        Swap(&arr[j], &arr[k]);
        j = k;
    }
    *p = *p - 1;
    return(r);
} // end of function
```